ResearchGate

See discussions, stats, and author profiles for this publication at: https://www.researchgate.net/publication/335649943

ArchLearner: leveraging machine-learning techniques for proactive architectural adaptation

Conference Paper · September 2019

DOI: 10.1145/3344948.3344962

CITATIONS
0

READS
252

2 authors:

Henry Muccini
Università degli Studi dell'Aquila
189 PUBLICATIONS **2,275** CITATIONS

SEE PROFILE

Karthik Vaidhyanathan
Gran Sasso Science Institute
6 PUBLICATIONS **2** CITATIONS

SEE PROFILE

Some of the authors of this publication are also working on these related projects:

Project ICT Living lab @ DISIM-UnivAQ View project

Project Collaborative MDSE View project

All content following this page was uploaded by Karthik Vaidhyanathan on 16 September 2019.

The user has requested enhancement of the downloaded file.

ArchLearner: Leveraging Machine-learning Techniques for Proactive Architectural Adaptation

Henry Muccini
University of L'Aquila
L'Aquila, Italy
henry.muccini@univaq.it

Karthik Vaidhyanathan
Gran Sasso Science Instute
L'Aquila, Italy
karthik.vaidhyanathan@gssi.it

ABSTRACT

Self-adaptation is nowadays considered as one of the possible solutions to handle the uncertainties faced by software at run-time. This is especially true in the case of IoT systems. These uncertainties can, in turn, affect the system QoS (Quality Of Service). In this tool demo, we present a machine learning driven proactive decision-making tool named ArchLearner, for aiding architectural adaptation. The tool enables the given IoT system to i) automatically identify the need for adaptation at an early stage; ii) perform automated decision making for generating the best adaptation strategy; iii) gather the feedback of the selected decision for continuous improvement. It also enables the architects/developers to i) visualize the adaptation process in near real-time; ii) specify the required configurations; iii) visualize the real-time QoS data.

CCS CONCEPTS

• **Computing methodologies** → **Machine learning**; • **Software and its engineering** → **Software architectures**.

KEYWORDS

IoT architectures, adaptive architectures, proactive decision making, machine learning

ACM Reference Format:
Henry Muccini and Karthik Vaidhyanathan. 2019. ArchLearner: Leveraging Machine-learning Techniques for Proactive Architectural Adaptation. In Proceedings of *European Conference on Software Architecture (ECSA), September 9–13, 2019, Paris, France.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3344948.3344962

1 INTRODUCTION

The uncertainties faced by software's are increasing day by day with software becoming more and more pervasive in nature. This is especially true in the case of IoT systems. Guaranteeing system QoS is considered as one of the most important challenges in architecting IoT systems [1] and uncertainties may in-turn have a high impact on QoS. Two such important QoS challenges are reducing energy consumption and data traffic [7] [1], particularly due to the expected increase in IoT devices which is estimated to be around 30 billion by 2020 [3].

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).
ECSA, September 9–13, 2019, Paris, France
© 2019 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-7142-1/19/09.
https://doi.org/10.1145/3344948.3344962

Self-adaptive solutions, especially proactive adaptation methods, can be used to handle these challenges as they can identify the need for adaptation at an early stage and ensure that the system maintains the expected QoS. Towards this end, we had proposed IoTArchML [11], an approach that uses machine learning techniques to forecast the expected QoS values. It then uses the forecasts to select the adaptation strategy using reinforcement learning techniques. For further information, please refer to [11].

Our work proposes ArchLearner, a tool that aids machine learning driven proactive architectural adaptation built on the top of IoTArchML. It uses the three self-adaptation patterns proposed in [12] to perform the adaptation. The tool leverages machine learning techniques implemented on an enterprise-grade big data stack. It enables the IoT system to i) automatically identify the need for adaptation at an early stage; ii) perform automated decision making for generating the best adaptation strategy; iii) gather the feedback of the selected decision for continuous improvement. It also provides a visual interface for i) defining the QoS thresholds; ii) visualizing the adaptation process; iii) specifying the forecasts and adaptation related configurations; iv) visualizing real-time QoS; v) adding/editing the machine learning related configurations.

The ArchLearner framework uses a combination of machine learning techniques to ensure that the system is able to foresee any possible QoS violations. It then performs adaptation by selecting the best adaption pattern to be used based on the forecasts. Thus, it ensures that the architecture of the system keeps improving with time.

2 THE ARCHLEARNER FRAMEWORK

ArchLearner enables software architectures to learn and improve continuously by leveraging machine learning techniques. It currently takes into account three prominent patterns for self-adaptation in IoT systems, namely Collect-Organize (CO), Synthesize Utilize (SU) and Synthesize-Command (SC) [12].

The ArchLearner framework is implemented using a layered architecture on an enterprise-grade big data stack. Figure 1 provides the data pipeline view of the framework. The framework architecture consists of five layers namely Architecture Modeling and Simulation, Event Queuing System, Stream Analytics, Persistence, and Presentation. The complete implementation of ArchLearner along with the source code and datasets can be found in [9]. Each of these layers map to the corresponding activities performed by ArchLearner. The layers can be summarized as follows.

1. Architecture and Simulation: This layer consist of the tools required for modeling the software architecture of the IoT system. ArchLearner, for now, uses the tool CupCarbon [2] for creating the architecture of the system. CupCarbon is an open-source IoT and

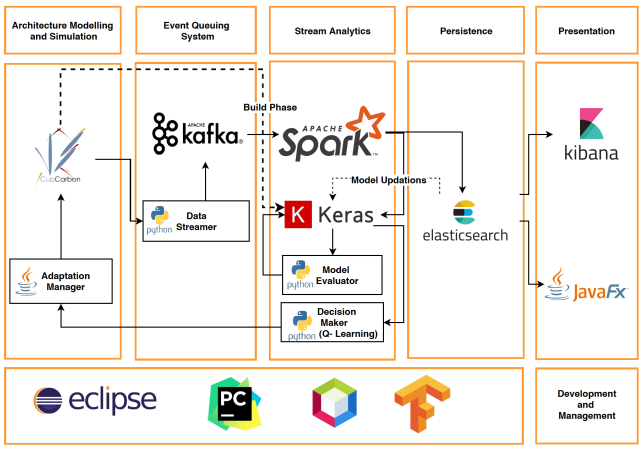

Figure 1: Data pipeline view of ArchLearner framework

Wireless Sensor Networks (WSN) simulator. It is considered as one of the best IoT simulators, especially for energy consumption [4]. It allows users to design and simulate IoT networks with sensors and other IoT components. In this layer, we design the architecture by creating components so as to facilitate the adaptation. For example, the CO pattern has additional controller components with respect to the SU and SC patterns. Subsequently, the architecture is simulated to generate data traffic and energy consumption logs. The ArchLearner tool may incorporate additional modeling tools to gather different QoS related data.

For performing the adaptation, ArchLearner uses an off-the-shelf component developed in Java. It receives adaptation decisions in the form of adaptation patterns that need to be applied as a configuration file from the decision maker component of the *stream analytics layer*. In order to achieve so, we modified the source code of CupCarbon. Further details can be found in [9].

2. Event Queuing System Layer: This layer is responsible for the processing of real-time events, in the form of energy and data traffic logs. It consists of two components namely, Data Streamer and Apache Kafka [6]. The role of the data streamer is to stream the logs produced by CupCarbon in real-time to Apache Kafka. It basically acts as the bridge between the CupCarbon and Kafka. ArchLeaner uses an off-the-shelf component developed using Python for achieving the same. In order for the predictions to happen in near real-time, the data needs to be analyzed and processed in near-real time. For this purpose, we use Apache Kafka which is specifically designed for distributed log processing. The data is further sent to the stream analytics layer.

3. Stream Analytics Layer: The responsibility of this layer is to perform QoS forecasts and generate dynamic architectural adaptation decisions based on the real-time data received from the *Event Queuing System layer*. It achieves this by using Apache Spark [13] in a streaming context, Keras [8] with TensorFlow backend to build the LSTM networks and the Decision Maker component which implements the Q-Learning based decision-maker algorithm using Python. Together they form the *Machine Learning Engine*. Further details on the algorithm can be found in [10].

The decision maker algorithm considers the fact that the system can enter into 9 possible states at any instant of time. These include *high energy & high data traffic (HE_HD), high energy & average data traffic (HE_AD), high energy & low data traffic (HE_LD), average energy & high data traffic (AE_HD), average energy & average data traffic (AE_AD), average energy & low data traffic (AE_LD), low energy & high data traffic (LE_HD), low energy & average data traffic (LE_AD), low energy & low data traffic (LE_LD)*. At any state, depending on the forecasts, the system can perform three actions which represent switching to one of the three self-adaptive patterns. Each action performed is given a reward, r based on the forecasts produced in the next time-step. This process of using q-learning ensures that any decision leading to higher energy/higher data traffic state will get a high penalty in the form of a high negative reward and this means as the time progresses the quality of adaptation improves.

During the machine learning model build phase, ArchLearner directly passes the data from the CupCarbon to Keras (represented by dotted lines in Figure 1) and this happens periodically so as to keep improving the model. The generated decisions in the form of a *.txt* file are then sent to the *Adaptation Manager* in the *Architecture and simulation layer*.

4. Persistence: The persistence layer is responsible for the storage and management of QoS data produced. The framework uses Elasticsearch [5] for achieving the same. It stores the time-series batch data obtained from the *Stream Analytics layer*. This data is further used for visualizations of the near real-time data traffic and energy consumption, as well as for further off-line training of the machine learning models (represented by the dotted lines).

5. Presentation: This layer is responsible for providing real-time visualizations of energy consumption and data traffic data. These visualizations can be further used to improve the configurations of the components in the architecture. ArchLearner provides two kinds of visualization. First, using Kibana [5] which allows creating multiple dashboards, based on the real-time QoS data from elasticsearch. Second, using a Java-FX based UI to provide real-time visualization of the selected adaptation process. The UI provides detailed views on how the adaptation is performed. It also allows stakeholders to define the respective configurations required for performing the adaptation, as well as the QoS thresholds, that the system has to maintain.

3 THE NDR CASE STUDY

Univaq Street Science is an event organized in the context of the European Researchers night (Notte dei Ricercatori (NdR)) in LÂquila. It is a scientific exhibition event organized by the University of L'Aquila. In this event, the research community and public are brought together from the afternoon until late at night to share a combination of entertainment and information. This event takes place throughout the entire city. As an example of a demonstrative scenario, we will take the NdR held in the city center, in which performances, lectures, demonstrations, workshops take place in its squares, main streets, and buildings.

3.1 Motivating Scenario

Our research group has been invited to develop an IoT based solution to improve the quality of the visiting experience during NdR.

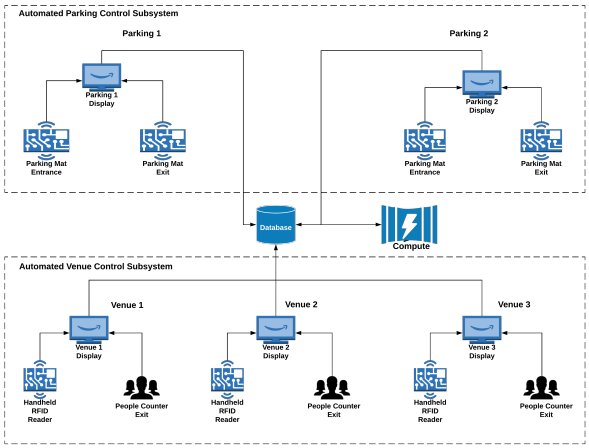

Figure 2: NdR Architecture

Without the loss of generality, In this paper, we focus on two such IoT services planned for NdR. These services are related to the automated venues management of 3 venues (capacities - 500, 200 and 200) and automated Parking lot control for 2 parking lots (capacities - 100 and 75). One important thing to note is that these parking lots and venues are created in an ad-hoc way since it is a one-day event and hence, the availability of external power sources is very limited.

We developed an IoT based solution (Figure 2) for automated parking control which uses battery-powered parking mats on the entrance and exit points of each of the parking lots. It then provides real-time notifications via a display on the parking lots as well as sends information to the central database which can be further sent to mobile applications. Similarly, for the automated venue management, we developed a solution which allows the venue manager to verify the entrance tickets using handheld RFID readers. It also allows them to perform automatic counting using people counters at the venue entrance and exit. In this way, the users can be redirected to different parking lots based on their venue preferences. However, one of the important constraints we have is to ensure that the sensors, such as the parking mats and the venue counters, being battery powered, need to operate in an energy efficient manner as they can run only for half a day. Even though charging can be performed, it might happen that during peak hours the charge goes down and plugging in a new sensor/charging the existing one can be costly in terms of crowd management. Moreover, we also need to ensure that the overall data traffic of the system is optimized since high data traffic can affect performance while little data traffic might cause issues with service accuracy. Hence, the challenge for us in this scenario is to create an architecture that can ensure an increased lifetime of the sensors ensuring that the overall data traffic is within an acceptable range.

3.2 Implementation Using ArchLearner

As explained in Section 2, we first modeled the architecture using CupCarbon for simulation. This was performed by specifying the software configuration for each of the component using senscript, the language offered by CupCarbon. It involves specifications such as the operational modes, frequency of data acquisition, etc. We then specified the hardware configuration by defining the parameters such as the communication protocol, battery specification, etc. For example, each of the sensor components uses WiFi protocol to transfer the data and uses batteries of capacity 19160 Joules. The process resulted in a total of 22 components as shown in Figure 2. During modeling, we also ensured that the senscript is flexible to switch between the patterns CO, SU and SC. We then simulated the architecture for a period of 10 days to generate the data traffic and energy consumption logs. This was further sent to the *Keras* component for creating the QoS predictor models (Build phase in Figure 1).

As and when the simulation is run, the logs are streamed in real-time through the *Data streamer* component to *Apache kafka*. We created one topic per sensor type to facilitate higher scalability and usability. In this way, every controller sends data to one topic "controller", sensors to topic "sensor" and so on.

This data is then pulled at a frequency of 10 seconds by *Apache spark* component in the streaming context. It keeps receiving the data from kafka and keeps aggregating the data until the data for 10 minutes is obtained which is then passed to *Keras* component for generating the energy and data traffic predictions.

The build phase results in the creation of two LSTM networks, one for energy forecasting and one for data traffic forecasting. These are further evaluated by the *Model evaluator* component for accuracy. During the normal execution, the aggregated 10 minute data are processed by the *Keras* component using the LSTM networks to generate the energy and data traffic forecast for the next 10 minutes. This energy and data traffic forecast are further sent to the decision maker component.

For the decision maker component, we use the thresholds $T = \{he = 14.0, le = 9.0, hd = 450, ld = 200\}$ where he represents high energy, le represents lower energy, hd, represents higher data traffic and ld represents lower data traffic. These thresholds are defined by considering the total energy available in batteries and the operational constraints we have from the case study. we use the rewards, $R = \{-10, -9, -8, -4, -3, -2, 0, 8, -1\}$ thus giving maximum reward for low energy and average data traffic state so that the total data traffic remains within the threshold. Further details regarding the model building and decision making process can be found in [10]. The generated decisions in the form of a .txt are then sent to the *Adaptation Manager* in the *Architecture and simulation layer*.

The batch data that is obtained from the *Stream Analytics layer* is stored in elasticsearch. We created an index, "archlearner" with a document name "energy_traffic". The id was set to auto-increment and we kept adding the data received in every batch. The time information was stored by using the type "timestamp" to facilitate time-series data representation.

We connected the elasticsearch data to Kibana by mapping the elasticsearch index, "archlearner" to Kibana cluster. Kibana allows us to create multiple dashboards and this option was used to create near real-time dashboards for the energy consumption data of different components in the system and data traffic of the system for different time intervals. We use the Java-FX based UI in the presentation layer to specify the different configurations required for running the ArchLearner tool. These include i) energy

and data traffic thresholds for performing adaption; ii) thresholds and reward values as required by the decision maker component; iii) CupCarbon project path to extract project related information; iv) Elasticsearch and Kibana host url; v) Apache Kafka and Spark host ip's; vi) machine learning model specific configurations such as number of neurons, number of hidden layers, path to save model, etc.

4 RESULTS

In this section, we describe the results of implementing the case study using our framework. We implemented ArchLearner on a Dell R730 CPU with Intel Xeon Processor comprising of 20 cores with CPU 256 GB of RAM. We used one compute node for running Apache Spark and one compute node to run Apache Kafka. Elastic search and Kibana were run on the third compute node with the fourth one being used for creating and testing machine learning models. This separation was done so as to mimic the real IoT data pipeline setup. In order to emulate a real-time scenario, we deployed the CupCarbon IoT simulator on a desktop machine running on Intel i5, 2.6-3.2 Ghz processor with 16 GB of RAM.

For building the machine learning model, we used the tool for configuring the model specifications such as the number of neurons, number of hidden layers, cross-validation split ratio, etc. We then ran the cupCarbon project for a period of 10 days, thus, resulting in a master dataset of energy consumption and data traffic logs each of size 1.7 GB. We used a cross-validation split ratio of 7:3 thus resulting in training and testing sets of 10080 and 4320 samples each. The LSTM models were trained using the training sets and the Normalized Root Mean Square Error (NRMSE) values on each of the testing sets were calculated. The LSTM for energy consumption prediction and data traffic forecast resulted in NRMSE values of 0.84 and 0.90 respectively.

We then calculated the effectiveness of ArchLearner by calculating the energy consumption and data traffic of the system while using the three patterns. This was performed by simulating each of the patterns as well as ArchLearner for a period of one day. We calculated the total energy consumed by the system from energy logs and the total data traffic from the execution logs. We then compared it with the energy consumption and data traffic of the system while using ArchLearner for a period of one day. The Kafka and spark configurations were specified in the ArchLearner UI and the simulation was performed. The results show that ArchLearner can offer approximately two hours of extra energy maintaining the required data traffic within the defined thresholds.

In order to calculate the quality of the adaptation, we logged the energy and data traffic consumption while using ArchLearner for every 2-hour interval. The results show that the energy and data traffic savings improved along with time. The reason for this behavior can be attributed to the ability of the decision maker to keep improving based on the feedbacks through rewards. The tool thus ensures that the system keeps learning and improving with time by moving towards the goal state of reducing energy consumption while keeping the data traffic within the limits.

We further evaluated the efficiency of ArchLearner by computing the number of adaptations performed in every 4-hour intervals and the average time taken for performing an adaptation. This was done by the option provided by the tool which allows generating the efficiency report. For each interval, the average number of adaptation performed amounts to 13 with the total number of adaptations amounting to 79. On the other hand, every interval amounts to 240 minutes and the approach continuously monitors every 10-minute data and this implies that the approach is able to intelligently perform the adaptation only when required based on the forecast. The report also shows that on an average the tool takes 0.023 seconds for performing the whole process. This speed is primarily due to the ability of the LSTM networks to provide the energy and data traffic forecasts in near real-time since it uses a pre-trained model. The periodic run of build phase ensures that this model is regularly kept up-to-date to provide accurate forecasts.

5 CONCLUSIONS AND FUTURE WORK

ArchLearner is a tool allowing architects/engineers to automate the process of proactive adaptation of the architecture through intelligent switching between the patterns and by leveraging the machine learning techniques like LSTM networks for QoS forecasting. We also were able to discuss the effectiveness and efficiency offered by the tool by applying the tool on NdR case study.

Future work include extending the tool for performing adaptation based on QoS parameters like response time, utilization, throughput, etc.; integrating a model checker so as to verify the correctness of the decision made by the decision maker component; and applying the tool on running instances for evaluations.

VIDEO

While this tool paper has been focusing on the ArchLearner architecture, the tool's behavior and its graphical UI components are illustrated in the video available at https://tinyurl.com/y6pf4vrt

REFERENCES

[1] Luigi Atzori, Antonio Iera, and Giacomo Morabito. 2010. The internet of things: A survey. *Computer networks* 54, 15 (2010), 2787–2805.

[2] Ahcène Bounceur. 2016. e. In *Proceedings of the International Conference on Internet of Things and Cloud Computing (ICC '16)*. ACM, New York, NY, USA, Article 1, 1 pages. https://doi.org/10.1145/2896387.2900336

[3] Cisco. 2018. CiscoIoT. http://tinyurl.com/y6bf2cgj

[4] 2 Juan F. Valenzuela-ValdĂĺs1 Cristina LĂşpez-PavÃşn1, Sandra Sendra1. 2018. Evaluation of CupCarbon Network Simulator for Wireless Sensor Networks. *Network Protocols and Algorithms* 10, 2 (2018), 453–475.

[5] Elasticsearch. 2019. Elasticsearch. https://elastic.co

[6] Apache Foundation. 2017. Apache Kafka. https://kafka.apache.org

[7] Sarra Hammoudi, Zibouda Aliouat, and Saad Harous. 2018. Challenges and Research Directions for Internet of Things. *Telecommun. Syst.* 67, 2 (Feb. 2018), 367–385. https://doi.org/10.1007/s11235-017-0343-y

[8] Keras. 2019. Keras : Deep Learning Library. https://keras.io

[9] Henry Muccini and Karthik Vaidhyanathan. 2019. IoTArchML. https://gitlab.com/mysat/iotarchml

[10] Henry Muccini and Karthik Vaidhyanathan. 2019. Leveraging Machine Learning Techniques for Automated Decision Making in Adaptive Architectures: A Technical Report. https://tinyurl.com/y445f45k

[11] H. Muccini and K. Vaidhyanathan. 2019. A Machine Learning-Driven Approach for Proactive Decision Making in Adaptive Architectures. In *2019 IEEE International Conference on Software Architecture Companion (ICSA-C)*. 242–245. https://doi.org/10.1109/ICSA-C.2019.00050

[12] Angelika Musil, Juergen Musil, Danny Weyns, Tomas Bures, Henry Muccini, and Mohammad Sharaf. 2017. *Patterns for Self-Adaptation in Cyber-Physical Systems*. 331–368. https://doi.org/10.1007/978-3-319-56345-9_13

[13] Matei Zaharia, Reynold S Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, Josh Rosen, Shivaram Venkataraman, Michael J Franklin, et al. 2016. Apache spark: a unified engine for big data processing. *Commun. ACM* 59, 11 (2016), 56–65.

View publication stats